

Hallucide

Hallucide vérifie les réponses d'une IA générative contre les sources officielles. La réponse est découpée en affirmations élémentaires ; chaque affirmation est confrontée mot pour mot au passage officiel récupéré (open data du Parlement, codes consolidés, data.gouv.fr). L'utilisateur ne reçoit jamais une affirmation non vérifiée présentée comme un fait : le verdict vient de la source, jamais du modèle.

Le moteur de vérification s'appelle Sentinel Guard (src/). C'est un pipeline déterministe : décomposition de la question, récupération des passages officiels, vérification verbatim, plancher de risque, validation humaine, journal d'audit. Spécification : docs/spec-v4.md. Statut d'implémentation : docs/STATUS.md.

Projet du hackathon de l'Assemblée nationale 2026, défi « IA et Hallucination » : voir hackathon-an-2026/DEFI.md.

Démarrage rapide

```
make setup          # première fois : crée .env et installe les
dépendances
cp .env.example .env # puis coller au moins une clé API (voir
Configuration)
make                # front de chat sur http://localhost:8770
```

make help liste les autres cibles (test, ui, stop, clean).

Configuration (.env)

Au moins une clé parmi :

Variable	Fournisseur
ANTHROPIC_API_KEY	Claude, modèle par défaut du front
MISTRAL_API_KEY	Mistral
GEMINI_API_KEY	Gemini

Sans clé, l'interface affiche « moteur non connecté ». Rien n'est simulé. .env n'est jamais versionné : ne jamais committer de vraie clé.

Arborescence

app/	front de chat DSFR + backend HTTP (python -m
app.server)	
src/	moteur Sentinel Guard, rangé par étape du
pipeline	
tests/	pytest (make test)
ui/	démonstrateur historique (python -m ui.server,
port 8765)	
mock/	maquette statique DSFR de référence
examples/	scripts exécutables par fonctionnalité du moteur
docs/	spec v4, statut, design, docs moteur
hackathon-an-2026/	fiche défi du hackathon

Architecture

```
Client → SentinelGuard.ask() → Orchestrator (décompose, récupère,
vérifie)
    |
    |— MultiSourceRetrievalProvider
    |   |— MoulineuseRetrievalProvider (normatif/
    parlementaire, MCP réel)
    |   |— DataGouvRetrievalProvider (donnée
    tabulaire, MCP réel)
    |   |— FileRetrievalProvider (fichier CSV/ZIP
    non tabulaire)
    |   |— verifier.py (contrôle verbatim
    déterministe, §7)
    |   |— triage.py (plancher de risque, §2)
    |   |— human_validation.py (validation humaine, §4 étape
    9)
    |   |— SovereignLogStore (journaux cloisonnés
    conformité/accès, §13.4)
```

Sources réelles branchées

- Moulineuse (mcp.code4code.eu) : articles de code consolidés, articles parlementaires, questions parlementaires (QE/QOSD/QG), commissions et mandats, recherche plein texte avec repli « pertinence non garantie », multi-saut réel.
- data.gouv.fr (mcp.data.gouv.fr) : données chiffrées tracées à la cellule exacte, statut `DONNÉE_TRACÉE`.
- Fichiers CSV/ZIP téléchargés directement pour les ressources non indexées par l'API tabulaire (couvre l'INSEE).
- Claude, Mistral et Gemini : LLM réels pour la décomposition et la génération de claims. Interchangeables, aucun ne juge sa propre fidélité.

Garanties (ce que le code prouve, pas le modèle)

- Une citation publiée AUTHENTIFIÉ existe mot pour mot dans la source officielle opposable récupérée (§7). Le vérificateur re-contrôle le cycle de vie : un texte abrogé n'est jamais AUTHENTIFIÉ.
- Le plancher de risque ne descend jamais sous « élevé » quand une condition déterministe est détectée (§2/INV-011) : référence inférée, troncature, pertinence non garantie, couverture insuffisante, sélection ambiguë, query partagée entre intentions.
- Une INTERPRÉTATION doit ancrer tous ses marqueurs de négation et toutes ses valeurs chiffrées dans la source, en plus du recouvrement lexical.
- L'opposabilité dérive du type de document et de son cycle de vie, jamais d'un flag de requête ni du modèle (INV-010).
- Mode document (v4, §7ter) : jamais de statut agrégé, chaque claim garde le sien (INV-015) ; l'omission silencieuse d'une unité structurelle bloque la publication (INV-017).
- Une intention à risque élevé n'est jamais publiée sans décision humaine explicite, capturée et journalisée (§4 étape 9).
- Le journal de conformité ne contient jamais la question posée ni une identité (§13.4), garde-fou vérifié par assertion.

Tests

```
make test
```

220 tests automatisés, dont le mode document v4 (INV-015/016/017, piège B5) et les commissions ciblées. Deux scénarios ont aussi été joués en direct contre les sources réelles (voir `docs/STATUS.md`) : prémisse fausse sans hallucination (NO_ANSWER), et passage authentique mais hors sujet bloqué par le plancher E1.

Exemples

Scripts exécutables dans `examples/`, un par fonctionnalité du moteur (Moulineuse et `data.gouv` ne demandent pas de clé) :

```
python examples/run_sentinel_guard.py
```